

TWIN ROTOR MIMO SYSTEM (TRMS)

Design, Modeling, and Control Implementation

Complete Project Documentation

System Specifications

Microcontroller: TM4C123GXL Tiva C

Motors: 1400KV BLDC $\times 2$ (5" main, 3" tail)

ESCs: SimonK 30A (pitch differential), Bidirectional 30A (yaw)

IMU: MPU6050 (I2C0)

Battery: 3S LiPo 11.1V 2200mAh 80C

Beam: 106 cm aluminum, 25 $\times$ 20 $\times$ 1mm hollow tube

1. Project Overview

1.1 Project Goals

The Twin Rotor MIMO System (TRMS) is a laboratory-scale model used to study control of cross-coupled multivariable dynamic systems. The project demonstrates classical and modern control techniques applied to a system that exhibits behavior similar to helicopter dynamics. The two primary control objectives are:

1. Maintain a desired pitch and yaw angle setpoint despite external disturbances
2. Reject step disturbances (e.g., a physical push on the beam) and return to setpoint within an acceptable settling time

1.2 Project Approach (Model-Based Design)

The project follows a model-based design workflow:

- Phase 1 — Manual data collection: PWM-to-angle relationships measured experimentally for both axes
- Phase 2 — System identification: Step-response data is logged via UART, and MATLAB's `tfit` fits a transfer function from input PWM to output angle
- Phase 3 — Controller design: PID controllers are tuned in MATLAB using the identified plant model, with Simulink simulation verifying disturbance rejection before deployment
- Phase 4 — Hardware deployment: Tuned controller gains are ported to the Tiva C as discrete-time PID code running at 100 Hz
- Phase 5 — Validation: Performance metrics (settling time, overshoot, steady-state error) are recorded and compared against simulation predictions

1.3 Why Model-Based Design Matters

Initial trial-and-error tuning of PID gains directly on hardware is unsafe and unreliable. A 1400KV BLDC motor at full throttle generates several Newtons of thrust on a 1-meter beam with significant moment arm. Aggressive gains can cause violent oscillation, structural damage, or motor burnout. By first validating gains in simulation against an identified plant model, the deployment to hardware proceeds with quantitative confidence rather than guesswork.

This addresses the core academic critique: a controller without a model is "driving without steering" — the control parameters have no theoretical basis and cannot be defended in a thesis or viva. The model-based approach ensures every gain has a physical justification.

2. Hardware Architecture

2.1 Mechanical Structure

The TRMS consists of a horizontal beam pivoted at its center. Each end of the beam carries a brushless motor with a propeller. The pivot allows two-degree-of-freedom rotation:

- Pitch (θ_p): Rotation in the vertical plane about the horizontal axis
- Yaw (θ_y): Rotation in the horizontal plane about the vertical axis

Geometric specifications (measured from the physical build):

Component	Dimension	Notes
Beam length	106 cm (1.06 m)	Full span between motors
Beam cross-section	25 mm × 20 mm	Hollow rectangular aluminum
Beam wall thickness	1 mm	
Beam mass (calculated)	246 g	From geometry × aluminum density
Pivot location	Center of beam	Distance to each motor: 53 cm
X-axis bearing diameter	8 cm	Pitch axis bearing
Y-axis bearing diameter	4 cm	Yaw axis bearing
Base diameter	30 cm	Tower mount

2.2 Electronic Components and Pin Mapping

Component	Specification	Connection
Microcontroller	TM4C123GXL Tiva C (ARM Cortex-M4F, 80 MHz)	USB power, I2C, PWM out
IMU	MPU6050 6-DOF (gyro + accel)	I2C0: PB2 (SCL), PB3 (SDA)
Pitch motor M1	1400KV BLDC, 5" propeller	ESC signal: PB6 (M0PWM0)
Pitch motor M2	1400KV BLDC, 5" propeller	ESC signal: PB7 (M0PWM1)
Pitch ESCs	SimonK 30A, unidirectional	Single rotation direction each

Component	Specification	Connection
Yaw motor	1400KV BLDC, 3" propeller	ESC signal: PB4 (M0PWM2)
Yaw ESC	Bidirectional 30A (Robu BD-ESC)	Reverses direction at 1500 μ s neutral
Battery	3S LiPo 11.1V nominal, 2200 mAh, 80C	Powers all 3 ESCs
UART	115200 baud over USB-Stellaris ICDI	PA0 (RX), PA1 (TX)
Buttons	SW1 (PF4) arm/zero, SW2 (PF0) E-stop	Internal pull-ups
LEDs	Red (PF1), Blue (PF2), Green (PF3)	Status indication

2.3 Pitch Differential Drive Architecture

Both pitch motors are unidirectional (SimonK firmware doesn't support reverse). Pitch control is achieved through differential thrust: the two motors push in opposite directions about the pivot. When the PID output is positive, M2 throttle increases above idle while M1 stays at or below idle. This produces a net upward torque about the pivot, raising the beam.

Differential drive has two key advantages over a single-motor approach:

- Bidirectional control with unidirectional motors — the beam can move both up and down despite each motor only spinning one direction
- Effective control gain is doubled — both motors contribute torque simultaneously, producing twice the moment for the same PWM differential

2.4 Yaw Bidirectional Drive Architecture

Yaw uses a single bidirectional ESC. The neutral point is at 1500 μ s (PWM count 938 at 50 Hz, 625 kHz PWM clock). Above neutral produces forward rotation; below neutral produces reverse rotation.

A characteristic of cheap bidirectional ESCs is an asymmetric dead-zone around neutral. Empirical measurements on this build:

- Forward dead-zone: 938 $\rightarrow$ 995 (motor inactive for 57 PWM counts above neutral)
- Reverse dead-zone: 938 $\rightarrow$ 910 (motor inactive for 28 PWM counts below neutral)
- Forward active range: 995 $\rightarrow$ 1250 (255 counts of usable authority)
- Reverse active range: 910 $\rightarrow$ 850 (60 counts of usable authority)

The dead-zones must be skipped in firmware — when the PID output crosses zero, the PWM output jumps directly from neutral to the active range edge (995 forward or 910 reverse) rather than wasting authority in the inactive zone. This is implemented in the `yawOutputToPWM()` function.

Note: Yaw control is asymmetric: forward authority is approximately 4 $\times$ greater than reverse. A single PID gain set is a compromise between the two directions. For a thesis-level project, this should be documented as a known limitation, with split forward/reverse gains as a future improvement.

3. PWM Configuration and Timing

3.1 Clock Tree

The Tiva C is configured for a 40 MHz system clock derived from the 16 MHz crystal through the PLL with SYSDIV_5. The PWM module clock is divided by 64 from the system clock:

```
SysCtlClockSet(SYSCTL_SYSDIV_5 | SYSCTL_USE_PLL |
                SYSCTL_OSC_MAIN | SYSCTL_XTAL_16MHZ);
SysCtlPWMClockSet(SYSCTL_PwMDIV_64);
```

System clock: 16 MHz × PLL / SYSDIV_5 = 40 MHz
 PWM clock: 40 MHz / 64 = 625 kHz
 1 PWM tick: 1 / 625 kHz = 1.6 μs
 PWM period: 625 kHz / 50 Hz = 12500 ticks (20 ms)

3.2 PWM Pulse Width to Tick Conversion

All ESC commands in the firmware are expressed in PWM ticks rather than microseconds, since PWM PulseWidthSet() expects ticks directly. The conversion is:

$$ticks = \mu s / 1.6$$

Pulse width (μs)	PWM ticks	Function
1000 μs	625	ESC arming pulse, motor off
1027 μs	642	Pitch idle (minimum spin point)
1500 μs	938	Yaw neutral (motor stopped)
1592 μs	995	Yaw forward minimum (active edge)
1456 μs	910	Yaw reverse minimum (active edge)
2000 μs	1250	Full throttle / full forward
1360 μs	850	Full reverse

4. Transfer Functions of the TRMS

4.1 Equations of Motion (Newton-Euler Approach)

The TRMS has two rotational degrees of freedom. Applying Newton's second law for rotation about each axis:

Pitch axis (rotation about horizontal axis through pivot):

$$J_p \cdot \ddot{\theta}_p + B_p \cdot \dot{\theta}_p + K_{g_p} \cdot \theta_p = K_{diff} \cdot u_p$$

where:

J_p = pitch axis moment of inertia (kg·m²)
 B_p = pitch axis viscous damping coefficient (N·m·s/rad)
 K_{g_p} = gravitational restoring stiffness (N·m/rad)
 K_{diff} = differential pitch motor gain (N·m per PWM count)
 u_p = differential PWM command (counts)

Yaw axis (rotation about vertical axis through pivot):

$$J_y \cdot \ddot{\theta}_y + B_y \cdot \dot{\theta}_y = K_y \cdot u_y$$

(no gravity restoring term – yaw rotates in horizontal plane)

where:

J_y = yaw axis moment of inertia (kg·m²)
 B_y = yaw damping coefficient
 K_y = yaw motor gain
 u_y = yaw PWM command (deviation from neutral, counts)

4.2 Computed Parameters from Build Geometry

All parameters are derived from the actual measurements of this TRMS build (Section 2.1):

Parameter	Symbol	Value	Source
Beam length	L_{beam}	1.06 m	Measured
Pivot to motor distance	l_{arm}	0.53 m	$L_{beam} / 2$
Beam mass	m_{beam}	0.246 kg	$\rho_{alum} \times \text{cross-section} \times \text{length}$
Motor + prop mass	m_{motor}	0.068 kg	Manufacturer + measured prop
Pitch moment of inertia	J_p	0.0612 kg·m ²	$(1/12) \cdot m_{beam} \cdot L^2 + 2 \cdot m_{motor} \cdot l_{arm}^2$
Yaw moment of inertia	J_y	0.0612 kg·m ²	Same geometry as pitch
Pitch damping	B_p	0.012 N·m·s/rad	Estimate (refine with sysID)
Yaw damping	B_y	0.005 N·m·s/rad	Estimate (refine with sysID)
Gravity stiffness	K_{g_p}	0.05 N·m/rad	Small balance offset

Parameter	Symbol	Value	Source
Pitch differential gain	K_diff	0.01526 N·m/count	$2 \times K_{\text{thrust}} \times l_{\text{arm}}$
Yaw thrust gain (forward)	K_y	0.00407 N·m/count	$K_{\text{thrust}} \times l_{\text{arm}}$

4.3 PITCH Transfer Function (Final)

Substituting the computed parameters into the equation of motion and taking the Laplace transform yields the pitch transfer function:

$$G_p(s) = \frac{\theta_{\text{pitch}}(s)}{U_p(s)} = \frac{0.01526}{0.06124 \cdot s^2 + 0.012 \cdot s + 0.05}$$

System characteristics:

- Natural frequency: $\omega_n = \sqrt{K_g/J} = \sqrt{0.05/0.0612} = 0.904 \text{ rad/s} \approx 0.144 \text{ Hz}$
- Damping ratio: $\zeta = B / (2 \cdot \sqrt{J \cdot K_g}) = 0.108$ — severely underdamped
- DC gain: $0.01526 / 0.05 = 0.305 \text{ rad per count} = 17.5^\circ \text{ per count}$
- Without active control, the beam will oscillate for many cycles after a disturbance before settling — this is exactly why a controller is required

4.4 YAW Transfer Function (Final)

The yaw axis has no gravitational restoring term, producing an integrator-type plant:

$$G_y(s) = \frac{\theta_{\text{yaw}}(s)}{U_y(s)} = \frac{0.00407}{0.06124 \cdot s^2 + 0.005 \cdot s}$$

$$= 0.00407 / [s \cdot (0.06124 \cdot s + 0.005)]$$

System characteristics:

- Type-1 system: one pole at the origin ($s = 0$)
- Second pole at $s = -0.0816 \text{ rad/s}$ (slow time constant $\approx 12 \text{ s}$)
- No steady-state position holding without active control — yaw will drift indefinitely
- Asymmetric in practice: forward gain is approximately 4× larger than reverse gain due to ESC mapping

4.5 State-Space Representation

Defining the state vector $x = [\theta_p, \omega_p, \theta_y, \omega_y]^T$, input $u = [u_p, u_y]^T$, output $y = [\theta_p, \theta_y]^T$:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -0.817 & -0.196 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -0.0816 \end{bmatrix} \quad B = \begin{bmatrix} 0 & 0 \\ 0.249 & 0 \\ 0 & 0 \\ 0 & 0.0664 \end{bmatrix}$$

$$C = [1 \ 0 \ 0 \ 0; \ 0 \ 0 \ 1 \ 0] \quad D = [0 \ 0; \ 0 \ 0]$$

Verifiable properties (run in MATLAB):

- $\text{rank}(\text{ctrb}(A, B)) = 4 \rightarrow$ fully controllable
- $\text{rank}(\text{obsv}(A, C)) = 4 \rightarrow$ fully observable
- Both pitch and yaw can be independently controlled and observed via MPU6050

4.6 Comparison with Reference Paper

A reference paper (Mundra et al., similar TRMS configuration) reported the following experimentally identified transfer function from black-box ARMAX identification:

$$G_{\text{ref}}(s) = \frac{(19.2 \cdot s^3 + 80.88 \cdot s^2 + 274.9 \cdot s + 468.7)}{s^3 + 2.785 \cdot s^2 + 5.284 \cdot s + 1.846}$$

Important observations about the reference TF:

- It is improper (3rd-order numerator over 3rd-order denominator), which is unusual for a strictly causal physical system. This suggests their identification included direct feed-through from input to output
- It mixes both axes plus cross-coupling and ESC dynamics into a single SISO model
- The mechanical specifications differ from this build (different beam length, motor specifications, counterbalance arrangement), so direct numerical comparison is not appropriate
- Their TF is useful for theoretical comparison but should not be substituted for system identification on this specific hardware

4.7 How to Verify the Transfer Function is Correct

A transfer function is a model — by definition an approximation of reality. Verification means quantifying agreement between model predictions and real measurements. Three levels of validation are used:

Level 1: Structural justification

The transfer function structure is derived from Newton's second law applied to rotational dynamics. The form (2nd-order for pitch, type-1 for yaw) is required by physics — it cannot be otherwise. This justifies the model's structure even before any experimental validation.

Level 2: Step response comparison

Apply the same step input to both the model (in MATLAB) and the real TRMS. Plot both responses on the same time axis. Compute the FIT% metric:

$$\text{FIT\%} = 100 \cdot (1 - \|y_{\text{real}} - y_{\text{model}}\| / \|y_{\text{real}} - \text{mean}(y_{\text{real}})\|)$$

Interpretation:

- > 80% Excellent fit, publishable quality
- 60-80% Good fit, suitable for control design
- 40-60% Mediocre fit, usable but expect manual tuning
- < 40% Poor fit, model structure may be wrong

For an undergrad TRMS project, FIT% > 60% is defensible.

Level 3: Cross-validation

Test the model on data not used to fit it. If FIT% remains within ~10% of the original fit, the model generalizes well and is trustworthy. A large drop indicates overfitting.

Level 4: Closed-loop verification

PID gains tuned on the identified model successfully achieve disturbance rejection on the real hardware. This completes the model-based design loop and demonstrates that the model is fit-for-purpose for controller design.

5. System Identification (Experimental Validation)

5.1 Why System Identification?

The physics-based model in Section 4 has the correct structural form, but several parameters are estimates: damping coefficients, motor thrust gains, gravitational restoring stiffness. Real damping depends on bearing wear, air drag on the specific beam shape, and motor electrical lag — none of which are known a priori.

System identification (sysID) measures the actual input-output behavior and fits a transfer function to the data. The identified TF then accurately reflects the real plant, including all unmodeled effects, and is suitable for controller design with confidence.

5.2 Step Response Experiment

The chosen identification approach is the step response method:

3. Apply a known step input (PWM jump from idle to a higher value)
4. Log the resulting angle response at 100 Hz via UART
5. In MATLAB, use `tfit()` to fit a transfer function to the data
6. Cross-validate on a different step magnitude to verify the model generalizes

5.3 Data Collection Procedure

The `trms_data_collector.c` firmware automates the data collection:

- Boots, settles IMU, calibrates gyro biases (2000 samples)
- After SW1 press, arms ESCs and ramps to idle
- Captures zero position (1-second average of pitch and yaw)
- After second SW1 press, runs an automated sequence of step tests
- Each test logs CSV data to UART with 10 ms sample period

Test sequence:

Test	Input	Duration	Purpose
Pitch step 30	Differential PWM = +30 counts	10 s	Small-signal response
Pitch step 60	Differential PWM = +60 counts	10 s	Mid-range response
Pitch step 90	Differential PWM = +90 counts	10 s	Large-signal (best SNR)
Yaw forward	PWM = 1130 (190 above neutral)	10 s	Forward yaw response
Yaw reverse	PWM = 870 (70 below neutral)	10 s	Reverse yaw response

5.4 MATLAB Workflow

The `trms_sysid.m` script processes the logged data through these stages:

7. Parse the multi-test log file using regex matching of test markers
8. Extract per-test CSV rows into structured MATLAB arrays
9. Visualize all step responses to verify data quality
10. Fit pitch transfer function using `tfest()` with both 2nd and 3rd order candidates
11. Cross-validate the chosen model on a different step magnitude
12. Fit yaw transfer function from the forward step data
13. Compare identified TFs against the physics-based predictions
14. Auto-tune PID controllers using `pidentune()` on the identified plant
15. Simulate disturbance rejection in closed-loop
16. Output PID gains formatted as C #defines for direct paste into firmware

5.5 Expected Outcomes

First-principles models for TRMS-class systems typically achieve 40-70% FIT against measurement. The gap is attributable to:

- Motor electrical time constant (causes ~50-100 ms lag, not modeled)
- ESC commutation delays at low throttle
- Aerodynamic drag nonlinearities (modeled as linear viscous damping)
- Gyroscopic precession at high RPM (cross-coupling, not modeled)
- IMU sensor noise and quantization

These effects are captured by the identified model since `tfest` fits to the actual response data, not to the idealized physics. The identified model is therefore preferred over the physics model for actual controller tuning.

6. PID Controller Design

6.1 Discrete-Time PID Algorithm

The controller is implemented in C as a discrete-time PID:

$$u(k) = K_p \cdot e(k) + K_i \cdot \sum e(i) \cdot dt + K_d \cdot (e(k) - e(k-1))/dt$$

where:

$e(k)$ = setpoint - measurement at sample k
 dt = sample period (10 ms)
 $\sum e(i) \cdot dt$ = cumulative integral, anti-windup clamped

6.2 PID Enhancements

- Derivative filter: First-order low-pass on derivative term to suppress sensor noise (cutoff $N = 8$ rad/s)
- Anti-windup: Integral clamping + back-calculation when output saturates
- Conditional integration: Integral only updates when $|\text{error}| < 5^\circ$ to prevent windup during large transients
- Deadband: Pitch 0.5° , yaw 3.0° to prevent hunting around setpoint
- Slew rate limiting: Maximum 15 PWM counts change per 10 ms cycle
- Output smoothing: Exponential filter on yaw output prevents derivative kicks from causing PWM stutter

6.3 Auto-Tuning in MATLAB

MATLAB's `piddtune()` function automatically computes K_p , K_i , K_d based on the identified plant model. The 'PIDF' structure includes a derivative filter, matching the firmware:

```
[C_pitch, info_p] = piddtune(G_pitch_id, 'PIDF');
[C_yaw, info_y] = piddtune(G_yaw_id, 'PIDF');
```

Reported metrics:

SettlingTime - time to within 2% of setpoint
 Overshoot - peak excess over setpoint (%)
 PhaseMargin - stability metric (target > 60°)
 CrossoverFreq - control bandwidth (rad/s)

7. 7-Day Action Plan with Learning Resources

7.1 Philosophy: Learn Just-in-Time

This plan integrates learning with execution. Each day, only the concepts needed for that day's task are learned. The total time investment is approximately 6 hours per day, with 2 hours of learning and 4 hours of execution.

Strategic principles:

- Pick ONE controller (PID), get it working end-to-end. Don't try LQR, fuzzy, and PID all in one week.
- Don't expand scope mid-week. Ship the demo first, then add features.
- Use MATLAB built-ins (pidtune, tfest) — don't reinvent the wheel.
- Manually port PID to C — Embedded Coder takes longer to set up than the port itself.
- If stuck for >2 hours, ask for help. Don't grind alone.

7.2 Day-by-Day Plan

Day 1 — Hardware Verification (3-4 hours)

Goal: Both pitch and yaw motors spin smoothly under known PWM commands.

Tasks:

17. Attach propeller to yaw motor
18. Run test_esc_motor.c to verify yaw responds in both directions
19. Verify pitch motors arm and reach idle
20. Test pitch differential drive (M2 up, M1 down) produces visible beam motion

Learning needed: None. Pure hardware verification.

Day 2 — MATLAB Setup + Data Collection (4-5 hours)

Goal: 5 step response CSV files saved on computer.

Learning resources (~2 hours):

- MATLAB Onramp at matlabacademy.mathworks.com — official 2-hour free course (skip if you have prior experience)
- YouTube: "matlab readtable csv import" tutorial (8 min)
- YouTube: "matlab plot subplot tutorial" (10 min)

Tasks:

21. Install MATLAB if needed; verify Control System Toolbox + System Identification Toolbox + Simulink are present (run 'ver' in MATLAB)
22. Flash trms_data_collector.c to Tiva
23. Open PuTTY at 115200 baud, enable session logging to file
24. Run the auto-test sequence (~80 seconds)
25. Save the log file as trms_log.txt

Day 3 — Transfer Function Identification (4-5 hours)**Goal: Identified TFs for pitch and yaw with FIT% > 60% on cross-validation.**

Learning resources (~3 hours):

- Brian Douglas — "What is a Transfer Function?" YouTube (8 min) — best intro available
- Brian Douglas — "Understanding Poles and Zeros" YouTube (12 min)
- MATLAB Tech Talk — "What Is System Identification?" YouTube (8 min)
- MATLAB — "Transfer Function Estimation" tutorial (10 min)

Tasks:

26. Run `trms_plant_model_v2.m` to see physics-based TFs
27. Run `trms_sysid.m` on the `trms_log.txt` file
28. Verify FIT% > 60% on identification data
29. Verify FIT% > 60% on cross-validation data (different step magnitude)
30. Save the workspace (saves `G_pitch_id`, `G_yaw_id`) for next day

Day 4 — PID Tuning + Simulation (3-4 hours)**Goal: Tuned PID gains; Simulink simulation showing disturbance rejection.**

Learning resources (~2 hours):

- Brian Douglas — "PID Control: A brief introduction" YouTube (8 min) — watch twice
- Brian Douglas — "Understanding PID Control" Parts 1-3 (~30 min total)
- MATLAB — "Designing a PID Controller using pidtune" YouTube (10 min)

Concepts to internalize:

- K_p = stiffness (larger = faster response, but oscillation if too high)
- K_i = removes steady-state error (but causes overshoot if too high)
- K_d = damping (but amplifies noise — needs filter)

Tasks:

31. In MATLAB: run `pidtune(G_pitch_id, 'PIDF')` and `pidtune(G_yaw_id, 'PIDF')`
32. Note the gains and verify settling time / overshoot are reasonable
33. Simulate impulse disturbance rejection in MATLAB
34. Optionally: build basic Simulink model with PID + plant + disturbance signal

Day 5 — Hardware Deployment (4 hours)**Goal: Closed-loop control running on TRMS with the tuned gains.**

Learning needed: None — engineering work.

Tasks:

35. Copy the gains from MATLAB into `main.c` #defines (`P_KP`, `P_KI`, `P_KD`, `Y_KP`, `Y_KD`)
36. Flash `main.c` to Tiva
37. FIRST TEST WITHOUT MOTOR POWER — verify UART telemetry shows sensible PID outputs

38. Power motors, set small setpoint (5° pitch), observe behavior
39. Increase setpoint gradually to 10° , then 15°
40. Note any oscillation or sluggishness for next day

Day 6 — Manual Tuning + Demo Recording (4-6 hours)

Goal: Disturbance rejection demo recorded on video.

Manual tuning rules:

- If oscillating: reduce K_p by 30%, then increase K_d by 50%
- If sluggish: increase K_p by 30%
- If steady-state error remains: add small K_i (start 10% of K_p)
- Always change one gain at a time

Tasks:

41. Iterate gain adjustments on hardware until response is smooth
42. Record 3 disturbance rejection trials per axis
43. Capture telemetry CSV from UART for plotting
44. Take video of beam settling after manual push

Day 7 — Documentation + Defense (4-6 hours)

Goal: Final report and viva preparation.

Tasks:

45. Write report sections: introduction, theory, methodology, results, conclusion
46. Include simulation vs hardware comparison plots
47. Include performance metrics table (settling time, overshoot, error)
48. Mention LQR/Fuzzy as future work
49. Practice the defense narrative (see Section 8.4 in this document)

8. Curated Learning Resources

8.1 The Single Most Important Resource

Brian Douglas's "Control System Lectures" YouTube channel is the gold standard for control theory intuition. He explains concepts more clearly than any textbook. Watching 8-10 of his videos provides all the theory needed for this project.

Channel: youtube.com/@BrianBDouglas

8.2 Resources by Topic

MATLAB Basics

- MATLAB Onramp — matlabacademy.mathworks.com (free, 2 hours, interactive)
- MATLAB Help YouTube channel — search any specific function
- MATLAB documentation: doc tfest, doc pidtune, doc lsim

Transfer Functions and Linear Systems

- Brian Douglas — "What is a Transfer Function?" (8 min)
- Brian Douglas — "Understanding Poles and Zeros" (12 min)
- Brian Douglas — "Time Response of Second-Order Systems" (10 min)
- MIT OCW 6.302 — Feedback Systems (free university lectures)

System Identification

- MATLAB Tech Talk — "What Is System Identification?" (8 min)
- MATLAB tfest documentation and examples
- Lennart Ljung — "System Identification: Theory for the User" textbook (advanced reference)

PID Control

- Brian Douglas — "PID Control: A brief introduction" (8 min)
- Brian Douglas — "Understanding PID Control" Parts 1, 2, 3 (~30 min)
- Brian Douglas — "PID Tuning" (15 min)
- MATLAB — "PID Tuning with pidtune" (10 min)

Simulink

- Simulink Onramp — matlabacademy.mathworks.com (free, 2 hours)
- MATLAB — "Build a PID Control System in Simulink" (12 min)
- MATLAB — "PID Tuning with Simulink Tuner" (8 min)

BLDC Motors and ESCs

- How2ElectronicsArticle — "BLDC Motor + ESC Working"
- Bidirectional ESC calibration video tutorials (search YouTube for your specific brand)

Tiva C Firmware

- TivaWare Peripheral Driver Library User Guide (PDF from TI)

- Sultan Qaboos University Tiva C tutorials (YouTube, free 30-video series)
- Texas Instruments E2E community for hardware-specific issues

8.3 What NOT to Learn (Saves Time)

With a 1-week timeline, skip these topics — they are not needed for the demo:

- Full control theory (root locus, Bode, Nyquist) — not required for PID with auto-tune
- LQR / state feedback design — only mention as future work
- Embedded Coder / auto code generation — manual port is faster
- Fuzzy logic theory — only mention as future work
- Nonlinear control theory — linearized model is sufficient for the operating range
- Full system identification with ARMAX — tfest is sufficient
- Kalman filtering details — complementary filter on MPU6050 is adequate

8.4 Daily Time Budget

Day	Learning	Execution	Total
Day 1 (hardware)	0 hours	3-4 hours	3-4 hours
Day 2 (MATLAB + data)	2 hours	2-3 hours	4-5 hours
Day 3 (sysID)	3 hours	1-2 hours	4-5 hours
Day 4 (PID design)	2 hours	1-2 hours	3-4 hours
Day 5 (deployment)	0 hours	4 hours	4 hours
Day 6 (tuning + demo)	0 hours	4-6 hours	4-6 hours
Day 7 (report)	0 hours	4-6 hours	4-6 hours
TOTAL	7 hours	19-27 hours	26-34 hours

9. Defense Narrative and Validation

9.1 Disturbance Rejection Demo Procedure

The primary demonstration of successful control:

50. Set both axes to a stable hold position (e.g., pitch = 0°, yaw = 0°)
51. Allow the system to settle until LEDs indicate "on target" (green)
52. Manually push the beam approximately 15-20° in pitch or yaw
53. Release the beam
54. Observe the controller drive the beam back to setpoint within 3-5 seconds
55. Verify minimal overshoot (< 20% of disturbance magnitude)
56. Verify settled steady-state error within 2°

9.2 Performance Metrics to Record

For the project report, capture these metrics from at least three trials per axis:

Metric	Definition	Target value
Settling time	Time from disturbance release to within 2° of setpoint	< 5 seconds
Overshoot	Peak excursion past setpoint after disturbance peak	< 20% of disturbance
Steady-state error	Average error after settling	< 2°
Rise time	Time to first cross 90% of step setpoint	< 1 second
Disturbance magnitude	Maximum angle deviation when pushed	10-20° (manual push)

9.3 Comparison: Simulation vs Hardware

In the project report, present side-by-side plots of:

- MATLAB-simulated step response of identified plant + tuned PID
- Hardware-measured step response of the same setpoint
- Both plots overlaid with time axis aligned

Quantify the simulation-to-hardware agreement using RMSE. A typical first-principles + system-identification project achieves 5-15° RMSE in the transient region. This is acceptable; the simulation predicts the qualitative behavior (rise time, overshoot, settling) within 30% of measured values, validating the model-based design approach.

9.4 Defending the Project — Question and Answer

Q: How do you know your transfer function is correct?

"I validated the model in three ways:"

"First, I derived it from first principles using Newton's laws and the measured geometry, so the structure is physically grounded — it's a 2nd-order system because the beam has inertia, damping, and gravitational restoring torque, exactly as the equations predict."

"Second, I compared the model's predicted step response against measurements from my actual TRMS. The FIT% was [your value]%, which is acceptable for control design."

"Third, I cross-validated by testing the same model against step responses I didn't use to fit it. The FIT% remained consistent, confirming the model isn't overfit."

"The model is not exact — no first-principles model is — but it captures the essential dynamics needed for PID controller design, which I verified by successful disturbance rejection on the real hardware."

Q: Why PID and not LQR or Fuzzy?

"PID is the appropriate complexity for the time available and the disturbance rejection requirement. LQR would require state estimation via Kalman filtering, which adds tuning complexity without significant performance gain for setpoint regulation. Fuzzy logic excels at nonlinear regions but my operating range is small enough that the linearized PID is sufficient. I've identified LQR and fuzzy controllers as future work in the report."

Q: Why didn't you use Embedded Coder?

"Embedded Coder requires significant toolchain setup that exceeds the project timeline. The PID controller is approximately 30 lines of C code — manually porting it from the Simulink-tuned design is faster than configuring auto-code generation. The model-based design loop (Simulink design → C deployment) is preserved; only the code generation step is manual."

Q: What's the limitation of your model?

"Three known limitations:

- Cross-coupling between pitch and yaw is not modeled — main rotor torque produces a yaw reaction not captured by the SISO transfer functions
- The yaw transfer function uses an averaged gain; in reality, forward gain is approximately 4× larger than reverse gain due to ESC dead-zone asymmetry
- Motor electrical time constant is not modeled, contributing to a small phase lag at higher frequencies

"For the operating range demonstrated, these limitations don't prevent successful control. Including them in the model would be the natural progression for a thesis-level extension."

10. Troubleshooting Reference

10.1 Yaw Motor Stutters

Symptom: Yaw motor makes irregular noise, doesn't spin smoothly.

Most likely causes, in order:

57. Propeller is missing — sensorless BLDC ESCs need rotor inertia and air load to commutate cleanly. Always test with the propeller attached.
58. ESC needs calibration — bidirectional ESCs require throttle endpoint calibration. Run the `calibrate_esc_v2.c` sketch with motor at MAX FORWARD when battery is connected.
59. Dead-zone PWM commands — sending PWM in the inactive zone (938-995 forward, 938-910 reverse) produces stutter. Use `yawOutputToPWM()` which jumps over the dead-zone.
60. PWM frequency mismatch — try changing `PWM_PERIOD` between 12500 (50 Hz), 6250 (100 Hz), and 3125 (200 Hz)

10.2 Pitch Motors Don't Reach Idle

- Pitch ESC arming pulse missed — ensure both pitch outputs are at `ESC_PITCH_MIN` (625 ticks) when battery first connected
- Battery voltage too low — 3S below 11.0V triggers ESC LVC protection
- ESC firmware mismatch — verify SimonK labeling

10.3 IMU Reads Wrong Angles

- Insufficient IMU settle time — use `IMU_SETTLE_MS` = 10000 or more
- Gyro bias drift — increase `GYRO_CAL_SAMPLES` to 2000 or 5000
- Vibration coupling — mount MPU6050 on foam
- I2C bus errors — verify pull-up resistors and shared ground

10.4 Closed Loop Oscillates

- K_p too high — reduce by 30%
- K_d too low — increase by 50%, watch for noise amplification
- Derivative filter cutoff too low — increase from 8 to 12 rad/s
- Integrator windup — verify `P_IMAX` is set

10.5 Beam Drifts to One Side at Idle

- Beam not balanced — add small mass to lighter side
- Pitch ESC idle differs between motors — adjust per-motor
- Asymmetric gravity — use `PITCH_FF` feedforward to compensate

10.6 Bidirectional ESC Won't Calibrate

- Calibration sequence requires MAX FORWARD signal BEFORE battery is connected

- Some Chinese ESCs (incl. Robu) don't support throttle calibration — they need a programming card
- Check for the 2-beep confirmation tone after the cell-count beeps

Appendix A: Project Files

All files referenced in this document:

Filename	Type	Purpose
main.c	C firmware	Operational closed-loop PID controller
trms_data_collector.c	C firmware	Open-loop step response data logger
test_esc_motor.c	C firmware	ESC + motor test sketch
calibrate_esc_v2.c	C firmware	Bidirectional ESC throttle calibration sketch
trms_sysid.m	MATLAB script	Parses log, fits TFs, tunes PID, exports gains
trms_plant_model_v2.m	MATLAB script	Physics-based model construction and PID tuning
validate_transfer_function.m	MATLAB script	Compares model prediction vs measured data
trms_log.txt	Log file	UART output from data collector (CSV format)
trms_identified_models.mat	MATLAB workspace	Saved TFs and tuned controllers

Appendix B: Final Transfer Functions Summary

PITCH:

$$G_p(s) = \frac{0.01526}{0.06124 \cdot s^2 + 0.012 \cdot s + 0.05}$$

$\omega_n = 0.904 \text{ rad/s} \mid \zeta = 0.108 \mid \text{DC gain} = 0.305 \text{ rad/count}$

YAW:

$$G_y(s) = \frac{0.00407}{0.06124 \cdot s^2 + 0.005 \cdot s}$$

$$= 0.00407 / [s \cdot (0.06124 \cdot s + 0.005)]$$

Type-1 system (integrator + slow first-order pole)
Forward/reverse gain ratio $\approx 4\times$ (asymmetric)

STATE-SPACE:

States: $x = [\theta_p, \omega_p, \theta_y, \omega_y]^T$
Inputs: $u = [u_p, u_y]^T$

Output: $y = [\theta_p, \theta_y]^T$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -0.817 & -0.196 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -0.0816 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 0 \\ 0.249 & 0 \\ 0 & 0 \\ 0 & 0.0664 \end{bmatrix}$$

$$C = [1 \ 0 \ 0 \ 0; \ 0 \ 0 \ 1 \ 0] \quad D = 0$$

Appendix C: Reference Datasheet Values

Quantity	Symbol	Value	Units
Beam length	L_beam	1.06	m
Pivot to motor distance	l_arm	0.53	m
Beam mass	m_beam	0.246	kg
Motor + prop mass	m_motor	0.068	kg
Pitch moment of inertia	J_p	0.0612	kg·m ²
Yaw moment of inertia	J_y	0.0612	kg·m ²
Pitch damping	B_p	0.012	N·m·s/rad
Yaw damping	B_y	0.005	N·m·s/rad
Gravity stiffness (pitch)	K_g_p	0.05	N·m/rad
Pitch differential gain	K_diff	0.01526	N·m per count
Yaw thrust gain (forward)	K_y_fwd	0.00407	N·m per count
Motor KV rating	KV	1400	RPM/V
Battery voltage	V_bat	11.1 (nominal)	V
PWM frequency	f_pwm	50	Hz
Control loop frequency	f_loop	100	Hz
IMU sample period	dt	0.01	s